

گزارش تحلیل و ارزیابی فنی کد پایتون

تحلیل جامع پروژه سیستم مدیریت آموزشی و پردازش داده‌ها

۱. مقدمه و ساختار کلی پروژه

پروژه بررسی شده یک سیستم مدیریت آموزشی شیء‌گرا به زبان پایتون است که امکاناتی نظیر مدیریت دانشجویان، دروس، ثبت نمرات، محاسبه معدل، مرتب‌سازی و تحلیل آماری نمرات را فراهم می‌سازد. این برنامه با رعایت اصول شیء‌گرایی (OOP) طراحی شده و از مفاهیمی چون وراثت (Inheritance)، کپسوله‌سازی (Encapsulation)، خصوصیات (Properties)، متدهای استاتیک و جادویی (Magic Methods) بهره می‌برد.

۲. تحلیل و ساختار کلاس‌ها

در این برنامه چهار کلاس اصلی تعریف شده است که هر یک وظایف مشخصی را بر عهده دارند:

کلاس Person (کلاس پایه)

این کلاس به عنوان کلاس پایه (Superclass) برای نگهداری اطلاعات عمومی افراد طراحی شده است.

- ویژگی‌ها (Attributes): شامل name (نام) و national_id (کد ملی).
- متدها: متد display_info() جهت بازگرداندن رشته‌ای شامل نام و کد ملی.

کلاس Course

این کلاس برای ساخت و نگهداری مشخصات دروس ارائه شده در سیستم استفاده می‌شود.

- ویژگی‌ها: course_code (کد درس)، title (عنوان درس) و credits (تعداد واحد).
- متدهای جادویی: پیاده‌سازی __str__ جهت ارائه نمایش متنی استاندارد از مشخصات درس.

کلاس Student (مشتق شده از Person)

این کلاس با استفاده از وراثت از کلاس Person مشتق شده و اطلاعات تخصصی دانشجو را نگهداری می‌کند.

- متغیر کلاسی (Class Variable): total_students که تعداد کل دانشجویان فعال در سیستم را شمارش می‌کند.
- خصوصیات (Properties):

○ gpa: محاسبه معدل وزن‌دار دانشجو به صورت دینامیک براساس ضرب نمره در تعداد واحد تقسیم بر مجموع واحدها.

○ total_credits: محاسبه مجموع واحدهای گذرانده شده.

- متدهای جادویی پیاده‌سازی شده:

○ __str__ و __repr__: جهت بازنمایی متنی و رسمی شیء دانشجو.

○ __eq__: جهت مقایسه دو دانشجو بر اساس شماره دانشجویی (student_id).

کلاس EducationalManager

کلاس اصلی مدیریت سیستم که وظیفه کنترل منطق برنامه، ذخیره‌سازی داده‌ها در ساختار دیکشنری، مرتب‌سازی و تحلیل‌های آماری را بر عهده دارد.

- ساختار داده: استفاده از دیکشنری برای students و courses جهت دسترسی سریع با کلید (کد دانشجویی / کد درس).

- مدیریت داده‌ها: متدهای افزودن/حذف دانشجو، ثبت درس، ثبت نمره و صدور کارنامه.

۳. الگوریتم مرتب‌سازی استفاده شده

در متد `EducationalManager.bubble_sort` از الگوریتم مرتب‌سازی حبابی (**Bubble Sort**) استفاده شده است. این متد به صورت یک `@staticmethod` پیاده‌سازی شده و از قابلیت تابع کلید (`key function`) پشتیبانی می‌کند. ویژگی مهم پیاده‌سازی: این الگوریتم به پرچم یا متغیر کمکی `swapped` مجهز شده است. در صورتی که در یک پیمایش کامل هیچ جابه‌جایی رخ ندهد، الگوریتم بلافاصله متوقف می‌شود (**Optimized Bubble Sort**). برنامه امکان مرتب‌سازی دانشجویان را بر اساس معیارهای مختلف ارائه می‌دهد:

- مرتب‌سازی بر اساس معدل (`gpa`)
- مرتب‌سازی بر اساس مجموع واحدها (`credits`)
- مرتب‌سازی بر اساس نام دانشجو (`name`)

۴. تحلیل پیچیدگی زمانی (**Time Complexity**)

در جدول زیر پیچیدگی زمانی بخش‌ها و متدهای مختلف برنامه بررسی شده است:

نام متد / عملیات	پیچیدگی زمانی	توضیحات و تحلیل فنی
افزودن / جستجوی دانشجو یا درس	$O(1)$	به دلیل استفاده از دیکشنری (<code>Hash Table</code>) برای ذخیره‌سازی.
محاسبه معدل (<code>Student.gpa</code>)	$O(K)$	K تعداد دروس دانشجو است؛ نیازمند پیمایش لیست دروس دانشجو.
مرتب‌سازی حبابی - بهترین حالت	$O(N)$	در شرایطی که آرایه از قبل مرتب باشد، به لطف پرچم <code>swapped</code> .
مرتب‌سازی حبابی - بدترین و متوسط	$O(N^2)$	N تعداد دانشجویان است؛ ناشی از دو حلقه تو در تو.
مرتب‌سازی کلی سیستم (<code>sort_students</code>)	$O(N^2 + N \cdot K)$	شامل محاسبه معدل/واحدها برای هر دانشجو و سپس اجرای مرتب‌سازی.
تحلیل آماری نمرات (<code>analyze_grades</code>)	$O(M)$	M تعداد کل نمرات ثبت شده در کل سیستم است (پیمایش نمرات + عملیات <code>NumPy</code>).

۵. تحلیل آماری انجام شده

عملیات تحلیل آماری در متد `analyze_grades` با بهره‌گیری از کتابخانه قدرتمند **NumPy** انجام می‌گیرد. ابتدا تمامی نمرات ثبت شده برای کلیه دانشجویان جمع‌آوری شده و به یک آرایه یک‌بعدی (`NumPy (np.array)`) تبدیل می‌شوند.

شاخص‌های آماری محاسبه‌شده عبارتند از:

- میانگین نمرات (**Mean**): با استفاده از `np.mean()` جهت سنجش سطح علمی کل سیستم.
- بیشترین و کمترین نمره (**Max / Min**): با استفاده از `np.max()` و `np.min()` برای تعیین دامنه تغییرات نمرات.
- انحراف معیار (**Standard Deviation**): با استفاده از `np.std()` برای ارزیابی پراکندگی نمرات حول محور میانگین.
- تعداد قبولی و مردودی: با استفاده از ماسک‌گذاری بولین و شرط شرطی روی آرایه `(grades_array >= passing_grade)` و محاسبه مجموع آن با `np.sum()`.